

FINFRONT SDK DESIGN LIBRARY

A CASE STUDY IN AI-NATIVE DESIGN SYSTEMS

FinFront Design Library

A case study for teams hiring a **Product Designer, AI Design Systems**

RICK HOLLANDSWORTH • PRINCIPAL EXPERIENCE DESIGNER

rick.hollandsworth@outlook.com • (201) 404-5031

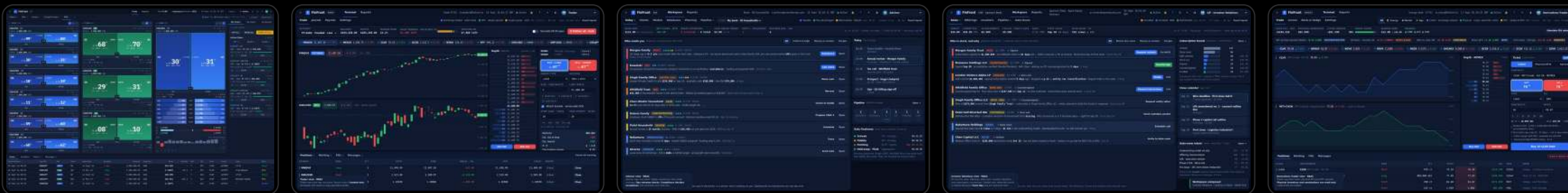
SEPTEMBER 2026

A real design problem, a self-directed prototype

FinFront SDK is a self-directed systems project: a headless React 19 / TypeScript component library and design system for trading, wealth-management, and alternative-investment platforms — five sector prototypes and a 75-component library, built to prove the system rather than to ship a product. It's a demo: every figure is synthetic, and no real capital moves through any of it.

I'm using it here because it's the clearest evidence I have of how I'd approach this role: I designed it, built it, documented it, and shipped the system alongside five live prototypes — working Claude-first the entire way. What follows is that evidence, screen by screen.

EVIDENCE, NOT ASSERTION — THE FIVE LIVE PROTOTYPES

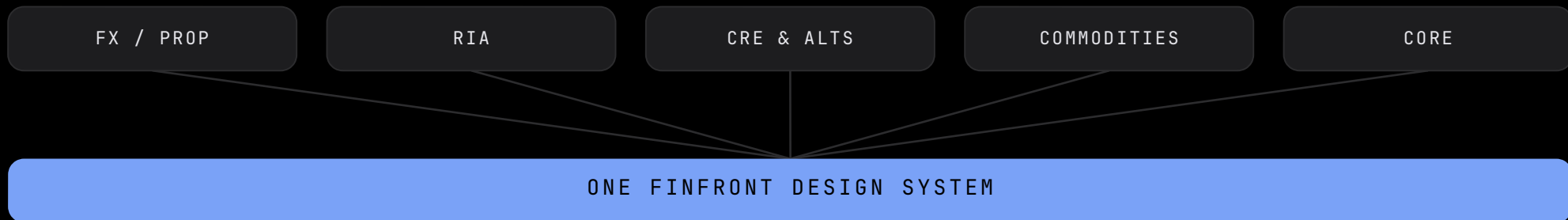


FinFront SDK is a self-directed prototype built for portfolio demonstration — not a licensed product. All data shown is synthetic.

A design system is infrastructure, or it's decoration.

You're looking for someone to make your design system the thing that makes speed and quality the same thing — across every surface you ship, with Claude as a first-class collaborator rather than an afterthought. That's the exact problem I set for myself on FinFront: one designer, five verticals, 75 components, and no budget for drift between what the library documents and what the five prototypes actually do.

The next few slides walk through the system I built to solve it — mapped directly to what this role is asking someone to own.



One SDK, five verticals, a real component discipline

FinFront is a headless component system for trading, wealth-management, and alternative-investment platforms — order books, order tickets, portfolio and deal dashboards, composed from one 75-component library across five live sector prototypes.

75

COMPONENTS

Documented in the design library.

5

VERTICALS

FX/Prop · RIA · CRE & Alts ·
Commodities · Core.

17

ROLES

Designed for, across four verticals.

1

TOKEN SYSTEM

Shared foundation every vertical inherits.

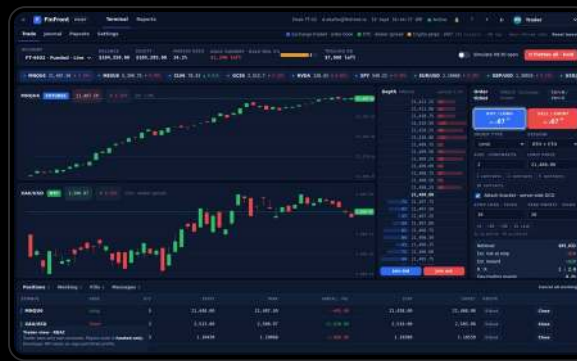
THE FIVE VERTICALS, LIVE



One brand, five verticals, zero room for drift

A trading terminal, an advisor workspace, and a GP capital-raise desk all need to agree — on color, on numbers, on what a severity flag means — without becoming the same app wearing five costumes. That's the same design-systems problem any growing product org runs into, just concentrated.

I solved it by refusing to treat the library as paperwork that trails the prototypes — pulling every token and class from real production CSS, so documentation can't drift from what it documents. The two screens below show why that discipline matters: the same brand, deliberately speaking two different visual languages.



PROP DESK — SEVERITY LADDER

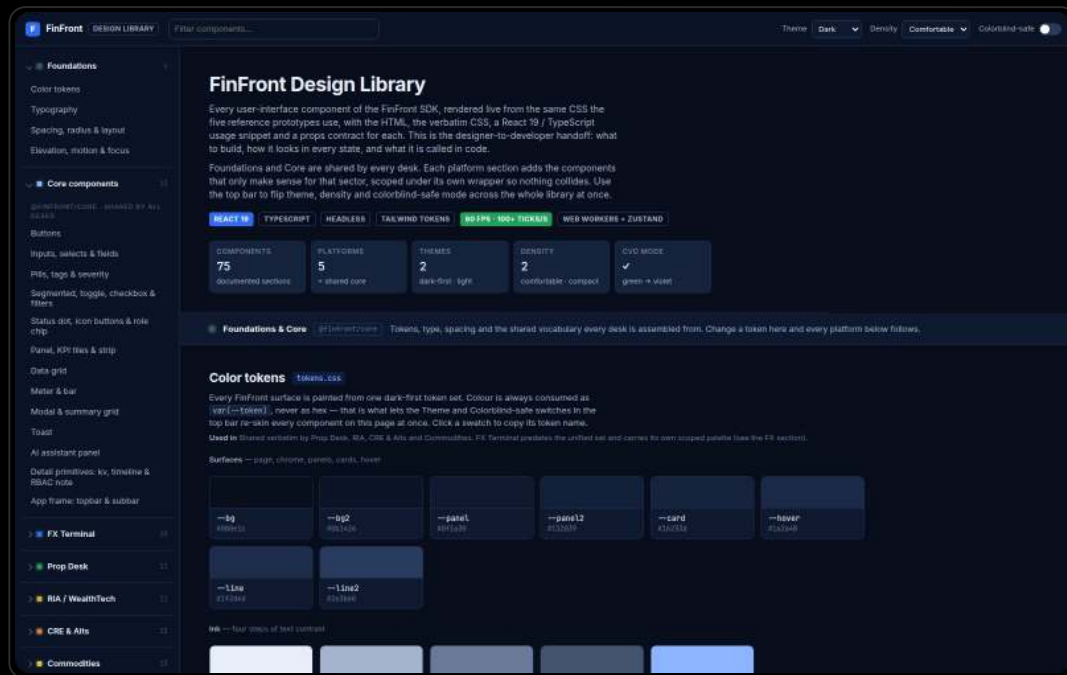


CRE — CAPITAL-LIFECYCLE COLOR

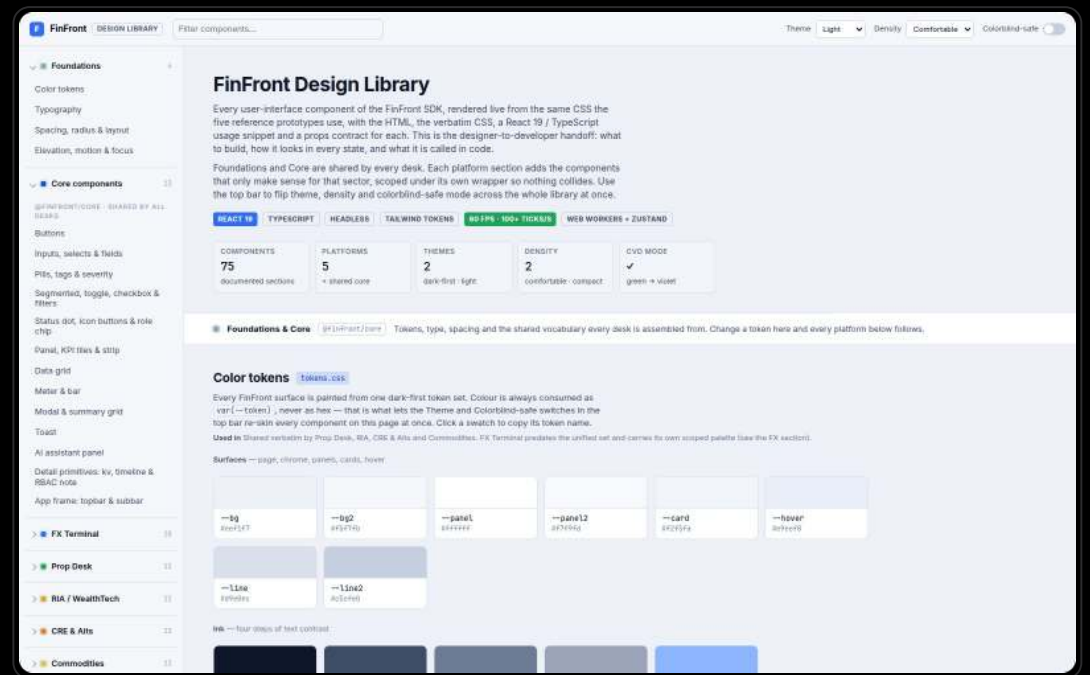
A library that's the source of truth, not a record of one

Every token, class, and component in the library is extracted directly from the five live prototypes' production CSS — not redrawn, not approximated. Dark and light aren't a toggle bolted on afterward; they're the same tokens resolving differently, everywhere, at once.

DARK



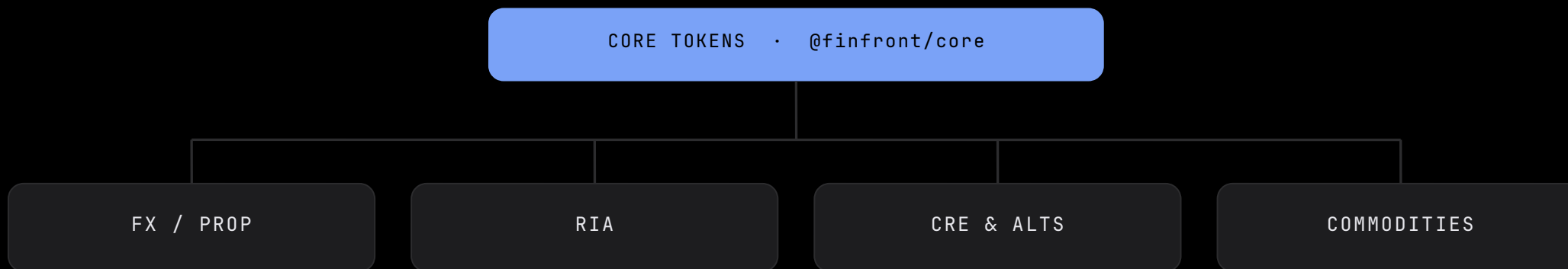
LIGHT



Shared foundations, platform-scoped components

FX/Prop, RIA, CRE & Alts, and Commodities share one token layer — color, type, the theming system — but scope their own component classes underneath it. RIA's severity chip is not CRE's severity chip, even though both compile from the same foundation.

I scoped every vertical's CSS under a `.plat-<name>` wrapper so nothing collides, and wrote the rule down so a sixth vertical follows the pattern instead of inventing a new one.



The same language, doing five different jobs

A high-frequency FX terminal is built for tick-by-tick execution depth. An advisor's RIA workspace is built for triage. A GP's capital-raise desk is built for pipeline velocity. None of that required reinventing color, type, or data semantics — only deciding what each surface is for, in both modes.

DARK MODE



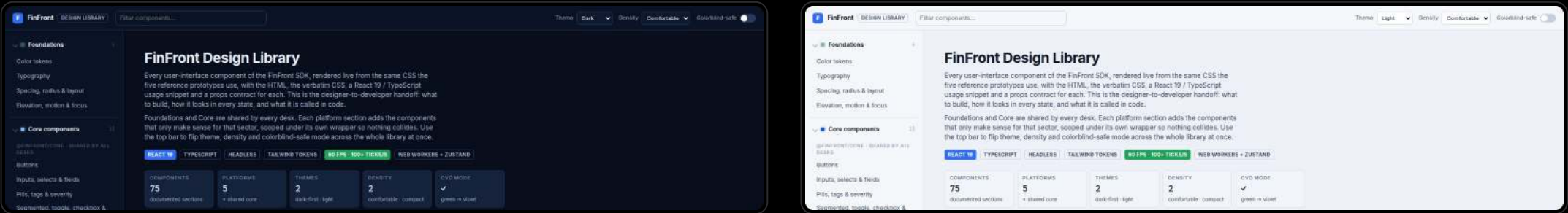
LIGHT MODE



Docs that behave like the product, not a record of it

Every component in the library is live and interactive, not a static frame. Toggle theme, density, or colorblind-safe mode from the library's own top bar, and every demo across all five verticals restyles at once — because it's driven by the same CSS custom properties the prototypes use.

SAME MARKUP, BOTH MODES — LIVE FROM THE LIBRARY



That's not a Storybook nicety. It's the difference between documentation a developer checks and documentation a developer trusts.

FULL INVENTORY, NOT JUST CORE ATOMS

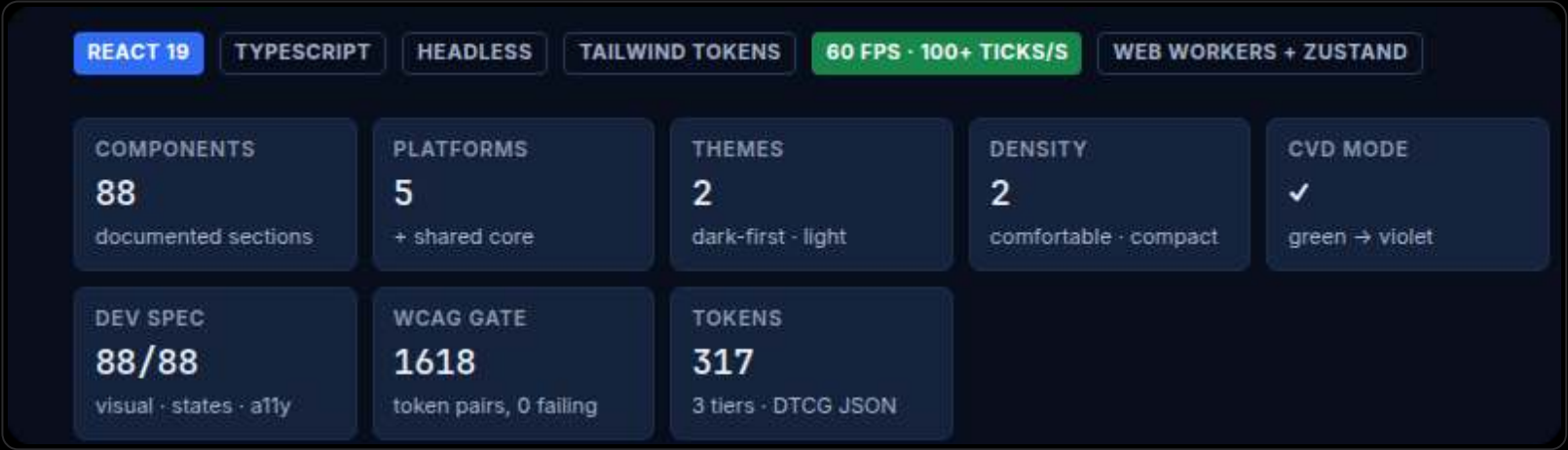
LIVE CONTROLS, NOT STATIC VARIANTS

REAL MARKUP, NOT SIMPLIFIED SNIPPETS

A hardening pass that turns “documented” into “buildable”

On September 29 I ran a dedicated hardening pass aimed at developer handoff: every one of the library's 88 documented sections — up from 82 — now ships a Spec tab: visual spec, every state in a table, behavior notes, edge cases, and accessibility requirements. ARIA coverage went from 17 attributes to roughly 2,790, and every token now traces to one DTCG source, gated at zero contrast failures across 1,618 WCAG-checked pairs.

LIVE FROM THE LIBRARY'S OWN HEADER — EVIDENCE, NOT ASSERTION



BEFORE → AFTER, THE SEPTEMBER 29 PASS

ANTICIPATING DEV QUESTIONS

C+ → A

DEV SPEC STRUCTURE

B- → A

TOKEN NAMING & EXPORT

D → A

Claude wasn't the intern. It was the pair.



WHAT I DIRECTED

- 01 Market and competitive research, synthesized.
- 02 Personas, UX strategy, IA — every vertical.
- 03 Visual language, design tokens, layout systems.
- 04 Component behavior, states, interaction rules.
- 05 The honesty of every deck, said plainly.



WHERE CLAUDE ACCELERATED ME

- 01 Fintech requirements, synthesized into docs.
- 02 React / TypeScript + CSS scaffolding, drafted.
- 03 The 75-section library, generated and QA'd.
- 04 Five verticals iterated in days, not months.

This is what “Claude-first” looks like in practice for a design system: not Claude designing, but Claude collapsing the distance between deciding something and proving it's true everywhere at once — which is precisely the leverage a founding design-systems hire needs.

Judgment calls, made and logged

Building a system alone means every cross-cutting decision is yours to catch. One shared severity ladder runs across Prop Desk, RIA, and Commodities. CRE & Alts doesn't get it: its capital lifecycle needed its own color family entirely, because forcing it onto the severity ladder would make two unrelated ideas compete for the same color.

I flagged that rather than quietly overloading a token — the instinct this role wants: notice when a pattern doesn't scale to a new vertical, and say so before it ships.

SEVERITY LADDER — PROP DESK · RIA · COMMODITIES

 CRITICAL  HIGH  MEDIUM  LOW

CAPITAL LIFECYCLE — CRE & ALTS ONLY

 UNCALLED  INVESTED  DISTRIBUTED

Two color families, deliberately kept apart — logged as a decision, not left as an accident.

Trust, verified before every publish

Every deck in this project — and every component in the design library — went through a real render-and-check pass: headless LibreOffice conversion, page-by-page visual QA, structural validation, before anything shipped.

A design system's credibility rests on this kind of unglamorous verification. I built it in as a standing habit on FinFront, not a special occasion reserved for launch day.

VERIFIED, NOT ASSUMED



Verified every publish

Rendered and checked headlessly before any file shipped.

Verbatim, not re-created

Every token pulled from real prototype CSS.

Logged, not silent

Judgment calls written down as they're made.

What you're asking for, and where I've already done it

- FOUNDDING MEMBER

Real say in how the system is built, staffed, led.

Sole designer on FinFront end to end — I set the architecture, made the calls, and logged every decision for continuity.

- OWN THE SYSTEM

Component library, tokens, docs, and code, end to end.

FinFront's library owns all four — extracted verbatim from five live prototypes, not re-created from memory.

- BUILD CLAUDE-FIRST

Claude produces on-system output by default.

Every layer — design, research synthesis, component generation, QA, documentation, even this deck — was built in direct collaboration with Claude.

- LEAD ON DESIGN

Influence decisions across teams, not one surface.

Made and can defend calls across five very different verticals sharing a single design language.

- A POV ON TOOLING

Help decide what the toolset looks like next.

Chose a live, theme-aware library over static frames — because static documentation lies the moment code changes.

I want to build the version of this that has teammates.

FinFront's design system is what one person, working Claude-first, can hold together across five verticals and 75 components. Yours is likely several products and a design-systems team that doesn't fully exist yet — that's the part I actually want next.

I'd bring the same habits at your scale: extract truth instead of approximating it, document live instead of documenting static, log judgment calls instead of burying them.

TODAY

1 designer, Claude-first. 5 verticals. 75 components. 17 roles designed for.

NEXT

A team that inherits the system — and the habits: extract truth, document live, log every call.

NEXT

//

Let's talk.

Rick Hollandsworth

Principal Experience Designer

`rick.hollandsworth@outlook.com` · (201) 404-5031